



CAPSTONE UMTAS

UNIVERSITY MODULAR TIMETABLE & ANALYTICS SYSTEM

TEAM: VIGIL

Wilmar Smit - u24584216

Aidan Dawson - u24593542

Johan Coetzer - u24564584

Marcel Stoltz - u24566552

Michael Tomlinson - u24569705

Team Contact: vigil.cs2025@gmail.com

Clients:



DNS.BUSINESS

TYTO 



Contents

1	Project Understanding	2
1.1	Problem Statement	2
1.2	Proposed Solution	2
1.3	Domain Model	3
2	System Architecture & Technology Stack	5
2.1	High-Level Architecture Overview	5
2.2	Technology Stack	6
3	Development Approach	8
3.1	Operational Modes	8
3.2	MVP-First Delivery	8
3.3	Quality Assurance	8
4	Team Capability	9
4.1	Team Members	9
4.2	Role Allocation & Strategy	11
4.3	Areas for Growth	11
5	Conclusion	12



1 PROJECT UNDERSTANDING

1.1 Problem Statement

South African universities publish timetables as static PDFs, forcing students to manually cross-reference pages to build personal schedules. This produces three compounding failures:

- **No automation:** Students manually resolve conflicts and build schedules with no tooling support — prone to error and significant time cost.
- **No personalisation:** No existing tool optimises for student preferences such as morning blocks, back-to-back classes, or minimum campus days.
- **No venue intelligence:** Venues are allocated on registration numbers, not planned attendance — producing over-crowded small venues alongside empty large ones. No South African university currently operates real-time venue demand analytics.

1.2 Proposed Solution

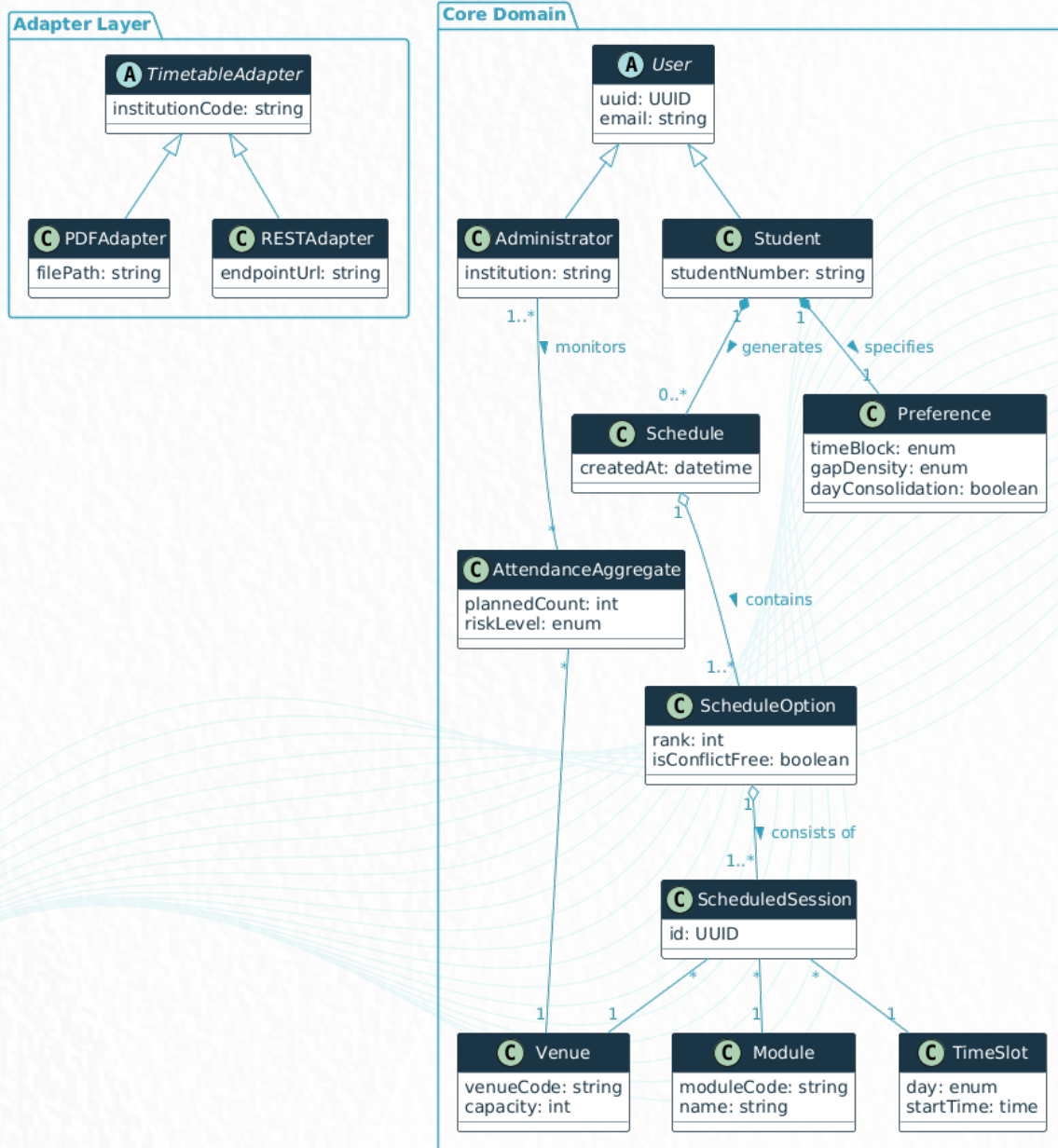
UMTAS addresses all three failures through a **university-agnostic Core-and-Adapter platform**. A timetable — regardless of institution — consists of the same fundamental elements: module codes, venues, time slots, and student groups. UMTAS builds a **generic core** around this normalised schema and wraps it with institution-specific **adapters**, meaning a new university requires only a new adapter with zero changes to core logic.

Development is sequenced in two phases:

1. **Core First:** The university-agnostic Nest.js API, FastAPI microservice, PostgreSQL schema, and authentication layer are built and stabilised before any adapter work begins. The core consumes a normalised internal timetable schema and is entirely indifferent to its data source.
2. **Adapters Second:** A University of Pretoria-specific pdf pLumber PDF adapter is built and validated against the stable core. A **mock REST API adapter** is additionally developed to demonstrate that the pattern supports non-PDF data sources — proving institutional extensibility without any core modification.



1.3 Domain Model



- **Adapter Layer (Abstraction):** Implements an **inheritance (IS-A) relationship** to decouple institutional data sources. It allows for specific PDF or REST implementations without modifying the generic core.
- **User Hierarchy:** Utilises a generalisation structure where **Students** and **Administrators** extend a base **User** class. This ensures shared attributes like authentication tokens are centralised while maintaining role-specific data.
- **Scheduling & Preference Engine:** Employs **composition** to link students to their unique **Preferences** (e.g., gap density, time blocks). This logic generates multiple **ScheduleOptions**, which are **aggregations** of individual **ScheduledSessions**.



- **Core Entities:** Defines the fundamental **associations** between **Modules**, **Venues**, and **TimeSlots**. These form the backbone of the normalised timetable schema used for conflict detection.
- **Privacy-First Analytics:** The **AttendanceAggregate** class represents a **UUID-dissociated** link between venue demand and time slots. This ensures that university administrators monitor only **anonymised attendance patterns**, maintaining strict **POPIA compliance**.



2 SYSTEM ARCHITECTURE & TECHNOLOGY STACK

2.1 High-Level Architecture Overview

The UMTAS architecture implements a modular Core-and-Adapter pattern. The Nest.js core manages central business logic, while a specialised Python microservice handles computationally intensive PDF extraction and scheduling optimisation. This separation ensures scalability and institutional flexibility.

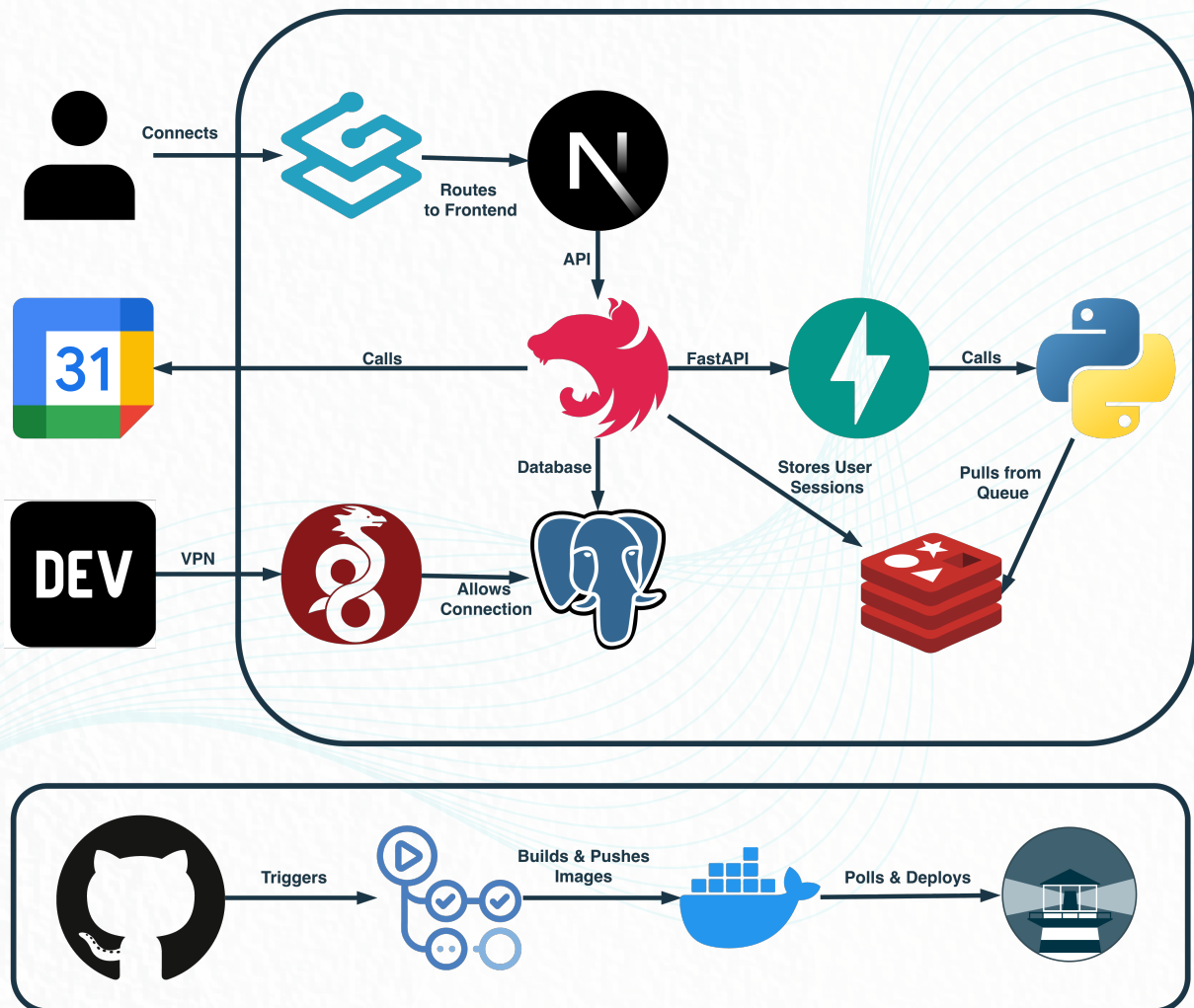


Figure 1: System Architecture and Service Interaction



2.2 Technology Stack

Frontend & Routing



Next.js (React + TypeScript)

Unified portal for students and admins; leverages SSR for performance and shared DTOs for type-safety.



Tailwind CSS

Rapid, mobile-first UI development with a minimal CSS footprint.



Traefik Edge Router

Handles automatic service discovery and SSL termination for all Docker containers.

Backend — Core API



Nest.js

Modular core managing the adapter layer; provides native OpenAPI documentation and robust dependency injection.



Passport.js & OAuth2

Secure authentication and scoped Google Calendar synchronization.



TypeORM

Strongly-typed relational mapping and versioned migrations for PostgreSQL.

Python Intelligence Microservice



FastAPI

High-performance async gateway for the optimization and extraction engines.



Optimization Stack

Utilizes **pdfplumber** for table extraction and **Google OR-Tools** for complex constraint-based scheduling.

Data & Messaging



PostgreSQL

Primary relational database for persistent timetable, preference, and user data.



**Redis**

Facilitates high-speed session storage and handles the BullMQ job queue between services.

**WireGuard**

Encrypted VPN tunnel for secure team access to management interfaces.

Infrastructure & CI/CD**GitHub Actions**

Automated CI/CD pipeline for testing, validation, and image builds on every PR.

**Docker & Watchtower**

Containerized environment ensuring dev-to-production parity with automated, zero-touch deployment.



3 DEVELOPMENT APPROACH

Team Vigil will operate under an **Agile Kanban** framework with a dual-mode operational tempo, tracked on a centralised GitHub Projects board.

3.1 Operational Modes

- **Cruise Mode (heavy academic periods):** One retrospective per week; mentor meeting monthly. Focus on lower-risk tasks: documentation, UI components, test coverage, and code review.
- **Crunch Mode (lighter academic / recess periods):** Three stand-ups per week; one retrospective per week; mentor meeting fortnightly. Major technical milestones — FastAPI microservice, UP PDF adapter, analytics dashboard, and Simulation Service — will be targeted during Crunch sprints.

3.2 MVP-First Delivery

Optional requirements (PWA push notifications, admin module management, AI schedule suggestions, trend export, LMS integration, group timetable sharing) are treated as planned but post-MVP. MVP scope:

1. Nest.js Core API: authentication, optimisation endpoint, Google Calendar OAuth2 sync, .ics export
2. FastAPI microservice: pdfplumber UP adapter, OR-Tools optimiser
3. PostgreSQL schema and TypeORM migrations
4. Redis: session store and BullMQ job queue
5. Next.js student portal: module selection, preference UI, schedule display
6. Next.js admin dashboard: venue heatmaps, over-capacity warnings
7. Traefik + Docker Compose deployment on Tyto Ubuntu server
8. Simulation Service: 20,000+ synthetic profiles under Locust load test

3.3 Quality Assurance

The GitHub Actions pipeline will execute unit and integration test suites across Nest.js and FastAPI on every pull request, with OpenAPI schema validation ensuring documentation never drifts from the implementation. The 100% PDF parsing accuracy and 0% schedule overlap success criteria will be covered by automated regression test suites running on every build.



4 TEAM CAPABILITY

Team Vigil comprises five University of Pretoria Computer Science students with complementary profiles across full-stack development, system architecture, DevOps, data engineering, and machine learning. While the team maintains specialised leads, a cross-functional approach is adopted, with all members (excluding the System Architect) contributing across the full stack to ensure high velocity and internal redundancy.

4.1 Team Members



LinkedIn



GitHub

Wilmar Smit

Team Lead & Integration Lead

Third-year Computer Science student (u24584216). Serving as the **Team Lead**, Wilmar acts as the primary coordinator between frontend and backend workstreams. He ensures architectural alignment across the stack and manages the integration of diverse components into a cohesive system.

His professional experience at **Tyto Insights** informs his approach to building scalable, production-ready systems. Wilmar focuses on technical "peace-keeping," facilitating communication between architects to ensure that changes in frontend state management or backend API contracts remain synchronised.



LinkedIn



GitHub

Michael Tomlinson

Lead Developer & System Architect

Third-year Computer Science student (u24569705). Currently a Software Developer Intern at **Tyto Insights**, with prior experience migrating legacy systems to modern stacks at **DCS Engineering**.

Michael serves as the **System Architect**, specialising in the UMTAS core stack: **PostgreSQL**, **TypeORM**, **Nest.js**, and **Next.js**. He is responsible for the **Core-and-Adapter** pattern implementation, ensuring the system remains university-agnostic. His [Tuks PDF Calendar](#) project provides the foundational domain expertise required for the project's complex PDF parsing challenges.





LinkedIn



GitHub

Johan Coetzer

Frontend Lead & Full-Stack Developer

Third-year Computer Science student (u24564584) and Software Developer Intern at **Tyto Insights** (**Skunkworks** division). As **Frontend Lead**, Johan manages the **Next.js** ecosystem, focusing on responsive component architecture and intuitive user experience.

His background in full-stack development allows him to bridge complex system logic with the interface, ensuring that the heavy data requirements of the UMTAS platform are presented through a high-performance, accessible dashboard.



LinkedIn



GitHub

Marcel Stoltz

DevOps Lead & Backend Specialist

Third-year Computer Science student (u24566552) and Software Developer Intern at **Tyto Insights** (**Skunkworks** division). Marcel leads the **DevOps and Infrastructure** workstream, specialising in **Docker** environments and automated deployment pipelines.

With a technical focus on system availability and CI/CD execution, he ensures the **Nest.js** backend and **PostgreSQL** services are optimised for high-performance delivery. His previous work at **Gendac** provides the backend versatility necessary to support the project's data-heavy microservices.



LinkedIn



GitHub

Aidan Dawson

Backend Developer & Integration

Third-year Computer Science student (u24593542) focused on **backend development and contract-driven integration**. Aidan is responsible for implementing core backend functionality while ensuring strict adherence to **API contracts** across all system components, validating **data integrity and PDF extraction accuracy**, and supporting **automated testing** to maintain reliable, consistent interactions between the frontend, backend, and external adapters within the UMTAS platform.



4.2 Role Allocation & Strategy

- **System Architecture & Core API:** **Michael Tomlinson** leads the implementation of the university-agnostic core, focusing on the backend logic and the TypeORM schema.
- **Frontend Ecosystem:** **Johan Coetzer** oversees the Next.js architecture and main system management for the UI/UX.
- **Integration & Coordination:** **Wilmar Smit** manages the interaction between frontend and backend architects, ensuring seamless data flow and conflict resolution.
- **DevOps & Infrastructure:** **Marcel Stoltz** manages CI/CD, Dockerization, and backend performance tuning for hardware constraints.
- **QA & Contract Enforcement:** **Aidan Dawson** upholds system contracts and reliability, managing automated testing and reliability metrics.
- **Full-Stack Versatility:** With the exception of the System Architect, all team members operate as full-stack developers. Additionally, a "test-your-own-unit" policy is enforced to ensure high code quality before integration.

4.3 Areas for Growth

- **Constraint Optimisation:** Implementing production-scale Google OR-Tools for multi-weight preference scheduling under concurrent load.
- **Predictive Analytics:** Training and validating the scikit-learn venue failure model on synthetic semester data for proactive maintenance alerts.
- **Privacy Engineering:** Ensuring POPIA and GDPR compliance through k-anonymity dissociation and secure cryptographic audit trails.



5 CONCLUSION

UMTAS represents a well-defined challenge requiring a balance of architectural rigidity and frontend flexibility. Team Vigil's approach — utilising a Core-and-Adapter pattern, a Core-first development sequence, and fully containerised deployment on the Tyto infrastructure — directly addresses the complex requirements outlined in the project specification.

Against the four defined success criteria, Team Vigil's plan delivers:

- **100% PDF parsing accuracy** via pdfplumber, verified against a ground-truth dataset through automated regression testing.
- **0% schedule overlap** guaranteed by OR-Tools constraint satisfaction, enforced by an automated test suite on every build.
- **Zero UUID leaks** across the anonymisation boundary, verified by a secure cryptographic audit trail.
- **20,000+ concurrent users** validated by the Locust-driven Simulation Service before a single real student logs in.

Three factors distinguish Team Vigil. First, our **professional synergy**: with four members currently or previously embedded at Tyto Insights and Gendac, the team operates with a pre-established workflow and direct familiarity with the client's infrastructure. Second, our **proven domain expertise**: the [Tuks PDF Calendar](#) project serves as deployed evidence that the central technical challenge of parsing UP schedules is already solved. Third, our **integration-first philosophy**: by dedicating specific roles to system architecture, contract enforcement, and frontend-backend synchronisation, we ensure that the UMTAS platform remains robust, university-agnostic, and production-ready.

Team Vigil is ready to begin on 20 April 2026.

vigil.cs2025@gmail.com

